



OWASP

THE OPEN WEB APPLICATION SECURITY STANDARD



*e-Governance
Standards & Guidelines*
Standards for e-Governance Applications

**2nd State Level Awareness Workshop for the
states of Andhra Pradesh & Telangana**



*Indraveni K
Centre for Development of Advanced Computing*

24th Mar 2022

WHAT IS OWASP?



- The Open Web Application Security Project® (OWASP) is a nonprofit foundation that works to improve the security of software.
- The OWASP Foundation launched on December 1st, 2001. Through community-led open-source software projects, hundreds of local chapters worldwide, tens of thousands of members, and leading educational and training conferences, the OWASP Foundation is the source for developers and technologists to secure the web.

OWASP BY THE NUMBERS

- Community-led open source software projects (200+)
- Over 200+ local chapters worldwide
- Tens of thousands of members
- Industry-leading educational and training conferences



OWASP TOP PROJECTS

-
- OWASP Application Security Verification Standard (ASVS)
 - OWASP Top Ten
 - OWASP Security Testing Guide
 - OWASP Security Testing Checklist
 - OWASP Testing Labs
 - OWASP Tools
 - OWASP Cheat Sheet Series
 - OWASP Security Knowledge Framework
 - OWASP SAMM

OWASP TOP PROJECTS

-
- OWASP Application Security Verification Standard (ASVS)
 - OWASP Top Ten
 - OWASP Security Testing Guide
 - OWASP Security Testing Checklist
 - OWASP Testing Labs
 - OWASP Tools
 - OWASP Cheat Sheet Series
 - OWASP Security Knowledge Framework
 - OWASP SAMM

OWASP ASVS



The Application Security Verification Standard is a list of application security requirements or tests that can be used by architects, developers, testers, security professionals, tool vendors, and consumers **to define, build, test and verify secure applications.**

- Latest ASVS 4.0.3
- 14 sections and over 260 controls
- Level 1 – Level 2 – Level 3

ASVS LEVELS



The Application Security Verification Standard defines three security verification levels, with each level increasing in depth.

- ASVS Level 1 is for **low assurance levels**, and is completely **penetration testable**
- ASVS Level 2 is for **applications** that contain **sensitive data**, which requires protection and is the recommended level for most apps.
- ASVS Level 3 is for the most **critical applications** - applications that perform high value transactions, contain sensitive medical data, or any application that requires the highest level of trust.

Each ASVS level contains a list of security requirements. Each of these requirements can also be mapped to security-specific features and capabilities that must be built into software by developers.

ASVS 4.0



	Applicability	Building			Building, Configuration, Deployment Assurance and Verification			Assurance and Verification	
Level 1	All apps		Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Penetration Testing	DAST
Level 2	All apps	Security Architecture and Reviews	Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Hybrid Reviews	SAST
Level 3	High Assurance	Security Architecture and Reviews	Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Hybrid Reviews	SAST
Legend		Acceptable	Suitable						

***level 1** that all applications should have,
level 2 for applications processing sensitive data,
level 3 for critical applications*

HOW TO USE ASVS STANDARD



- **Use as a metric** - Provide application developers and application owners with a yardstick with which to assess the **degree of trust** that can be placed in their Web applications,
- **Use as guidance** - Provide guidance to security control developers as to what to build into security controls in order to satisfy application security requirements, and
- **Use during procurement** - Provide a basis for specifying application security verification requirements in contracts.

ASVS requirement lists are made available in CSV, JSON, and other formats which may be useful for reference or programmatic use. - <https://github.com/OWASP/ASVS>

HOW TO USE ASVS STANDARD



- ❑ As Detailed Security Architecture Guidance
- ❑ As a Replacement for Off-the-shelf Secure Coding Checklists
- ❑ As a Guide for Automated Unit and Integration Tests
- ❑ For Secure Development Training
- ❑ As a Driver for Agile Application Security
- ❑ As a Framework for Guiding the Procurement of Secure Software

ASVS LEVELS ACHIEVEMENT



- Level 1 - First steps, automated, or whole of portfolio view
 - An application achieves ASVS Level 1 if it **adequately defends against application security vulnerabilities that are easy to discover** and included in the OWASP Top 10 and other similar checklists.
 - Threats to the application will most likely be from attackers who are using **simple and low effort techniques** to identify easy-to-find and easy-to-exploit vulnerabilities.
- Level 2 – Most applications
 - An application achieves ASVS Level 2 if it adequately defends against most of the risks associated with software today
 - Threats to Level 2 applications will typically be **skilled and motivated attackers focusing** on specific targets using tools and techniques that are highly practiced and effective at discovering and exploiting weaknesses within applications.
- Level 3 - High value, high assurance, or high safety
 - ASVS Level 3 is the **highest level of verification** within the ASVS
 - An application at ASVS Level 3 requires **more in-depth analysis or architecture, coding, and testing** than all the other levels

HOW TO USE ASVS STANDARD



Organizations are strongly encouraged to look deeply at your unique risk characteristics based on the **nature of your business** and based upon that **risk and business requirements** determine the appropriate ASVS level.

GLIMPSE ON ASVS 4.0 CONTROLS



V14.4 HTTP Security Headers Requirements

#	Description	L1	L2	L3	CWE
14.4.1	Verify that every HTTP response contains a content type header specifying a safe character set (e.g., UTF-8, ISO 8859-1).	✓	✓	✓	173
14.4.2	Verify that all API responses contain Content-Disposition: attachment; filename="api.json" (or other appropriate filename for the content type).	✓	✓	✓	116
14.4.3	Verify that a content security policy (CSPv2) is in place that helps mitigate impact for XSS attacks like HTML, DOM, JSON, and JavaScript injection vulnerabilities.	✓	✓	✓	1021
14.4.4	Verify that all responses contain X-Content-Type-Options: nosniff.	✓	✓	✓	116

GLIMPSE ON ASVS 4.0 CONTROLS



V10.2 Malicious Code Search

Malicious code is extremely rare and is difficult to detect. Manual line by line code review can assist looking for logic bombs, but even the most experienced code reviewer will struggle to find malicious code even if they know it exists.

Complying with this section is not possible without complete access to source code, including third-party libraries.

#	Description	L1	L2	L3	CWE
10.2.1	Verify that the application source code and third party libraries do not contain unauthorized phone home or data collection capabilities. Where such functionality exists, obtain the user's permission for it to operate before collecting any data.		✓	✓	359
10.2.2	Verify that the application does not ask for unnecessary or excessive permissions to privacy related features or sensors, such as contacts, cameras, microphones, or location.		✓	✓	272
10.2.3	Verify that the application source code and third party libraries do not contain back doors, such as hard-coded or additional undocumented accounts or keys, code obfuscation, undocumented binary blobs, rootkits, or anti-debugging, insecure debugging features, or otherwise out of date, insecure, or hidden functionality that could be used maliciously if discovered.			✓	507

GLIMPSE ON ASVS 4.0 CONTROLS



V1.2 Authentication Architecture

When designing authentication, it doesn't matter if you have strong hardware enabled multi-factor authentication if an attacker can reset an account by calling a call center and answering commonly known questions. When proving identity, all authentication pathways must have the same strength.

#	Description	L1	L2	L3	CWE
1.2.1	Verify the use of unique or special low-privilege operating system accounts for all application components, services, and servers. (C3)		✓	✓	250
1.2.2	Verify that communications between application components, including APIs, middleware and data layers, are authenticated. Components should have the least necessary privileges needed. (C3)		✓	✓	306
1.2.3	Verify that the application uses a single vetted authentication mechanism that is known to be secure, can be extended to include strong authentication, and has sufficient logging and monitoring to detect account abuse or breaches.		✓	✓	306
1.2.4	Verify that all authentication pathways and identity management APIs implement consistent authentication security control strength, such that there are no weaker alternatives per the risk of the application.		✓	✓	306

OWASP TOP PROJECTS

-
- OWASP Application Security Verification Standard (ASVS)
 - **OWASP Top Ten**
 - OWASP Security Testing Guide
 - OWASP Security Testing Checklist
 - OWASP Testing Labs
 - OWASP Tools
 - OWASP Cheat Sheet Series
 - OWASP Security Knowledge Framework
 - OWASP SAMM

OWASP TOP TEN



OWASP Top Ten is a publicly shared standard awareness document for developers of the ten most critical application security vulnerabilities, according to the Foundation.

The OWASP Top TEN list is developed by application security experts worldwide and is updated every couple of years. It aims to educate companies and developers on how to minimize application security risks.

Various OWASP Top Tens are:

- OWASP Top Ten Web Application Security Risks
- OWASP Mobile Top Ten
- OWASP API Security Top Ten
- OWASP IoT Top Ten
- OWASP Desktop App Security Top Ten
- OWASP Cloud-Native Application Security Top Ten
- OWASP Top 10 Proactive Controls

TOP 10 WEB APPLICATION SECURITY RISKS



A01:2021-Broken
Access Control

A02:2021-
Cryptographic
Failures

A03:2021-
Injection

A04:2021-
Insecure Design

A05:2021-
Security
Misconfiguration

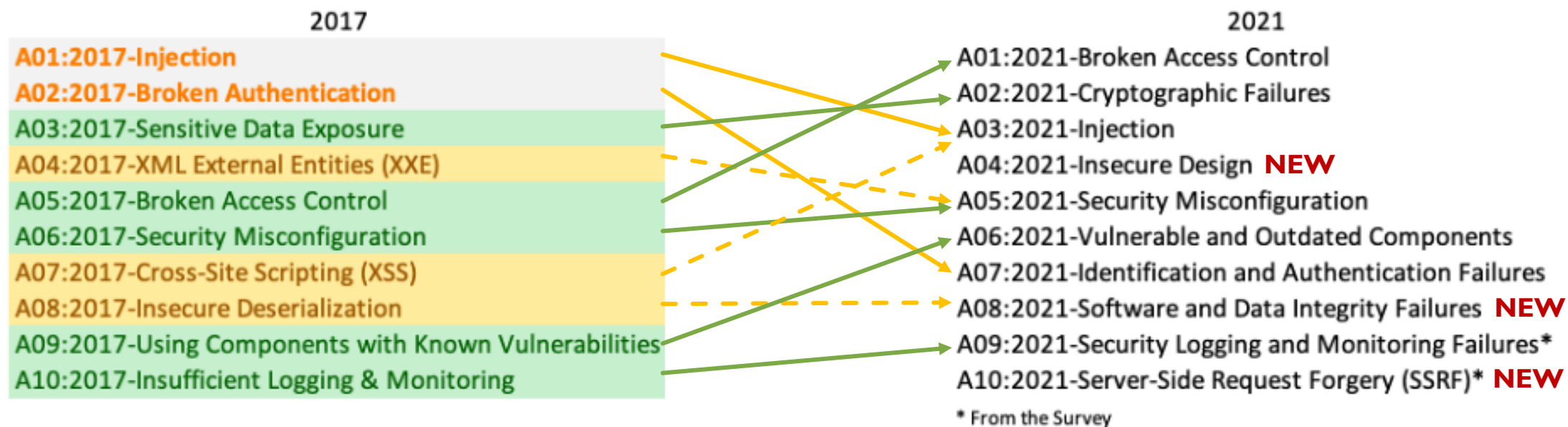
A06:2021-
Vulnerable and
Outdated
Components

A07:2021-
Identification and
Authentication
Failures

A08:2021-
Software and
Data Integrity
Failures

A09:2021-
Security Logging
and Monitoring
Failures

A10:2021-Server-
Side Request
Forgery



TOP 10 WEB APPLICATION SECURITY RISKS MAPPING

Broken Access Control:

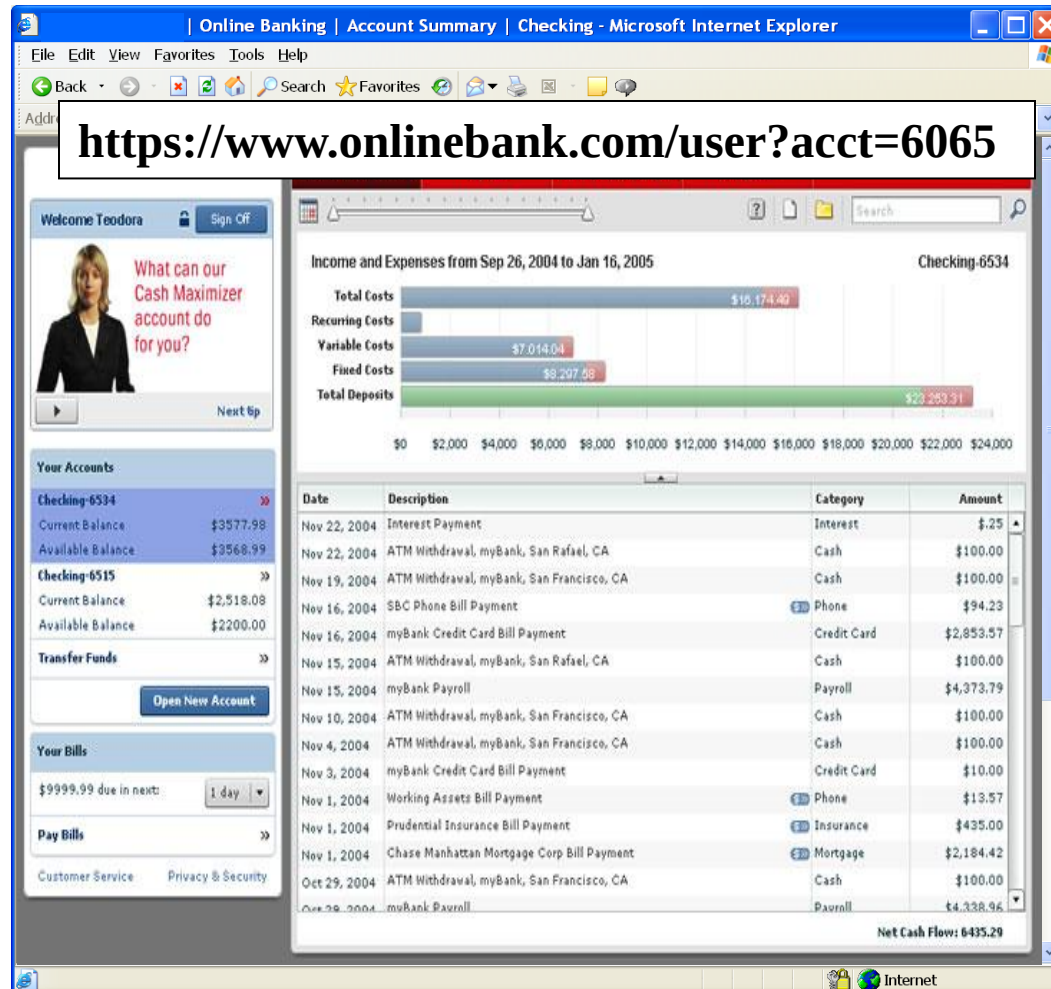
Access control enforces policy such that users cannot act outside of their intended permissions. Failures typically lead to unauthorized information disclosure, modification or destruction of all data, or performing a business function outside of the limits of the user.



Common access control vulnerabilities include:

- Bypassing access control checks by modifying the URL, internal application state, or the HTML page, or simply using a custom API attack tool
- Allowing the primary key to be changed to another's users record, permitting viewing or editing someone else's account.
- Elevation of privilege. Acting as a user without being logged in, or acting as an admin when logged in as a user.
- Metadata manipulation, such as replaying or tampering with a JSON Web Token (JWT) access control token or a cookie or hidden field manipulated to elevate privileges, or abusing JWT invalidation
- CORS misconfiguration allows unauthorized API access.
- Force browsing to authenticated pages as an unauthenticated user or to privileged pages as a standard user. Accessing API with missing access controls for POST, PUT and DELETE.

Broken Access Control - Scenario I

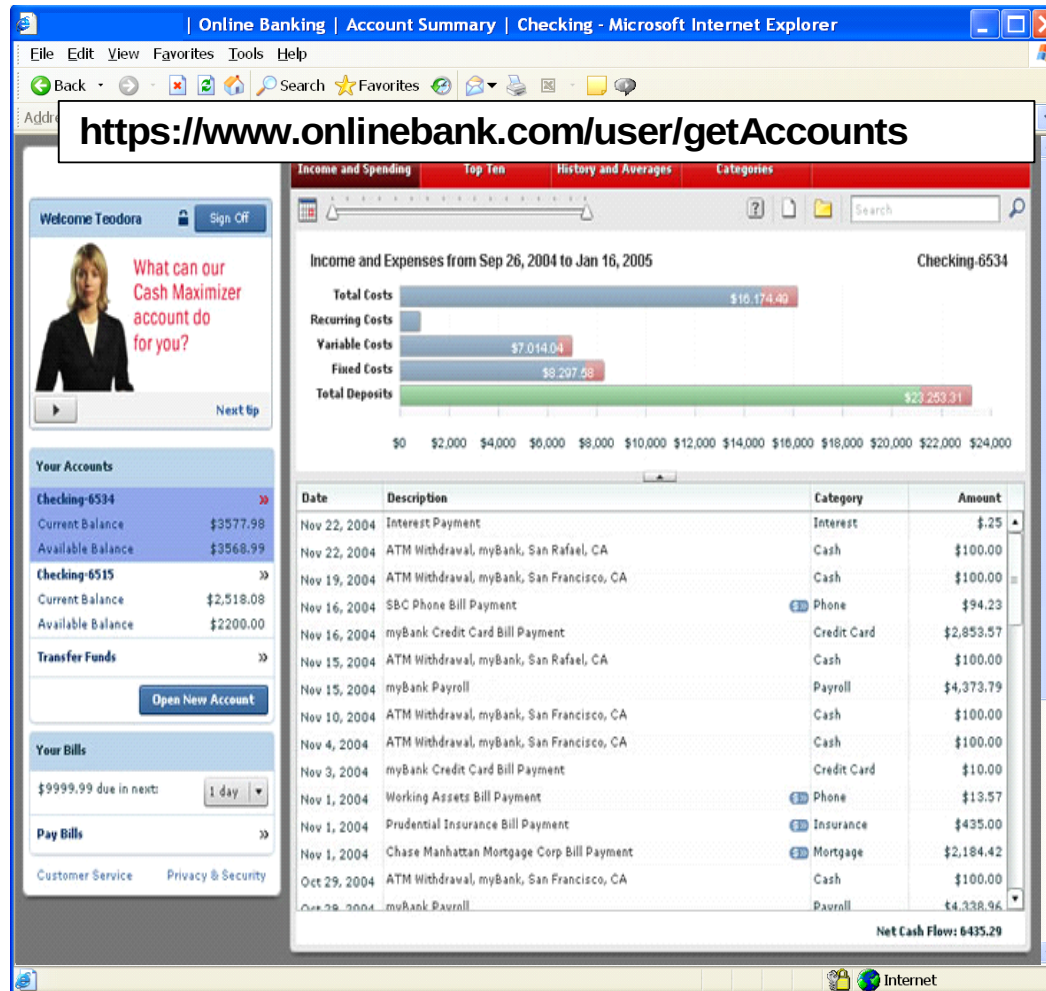


■ Attacker notices his acct parameter is 6065
?acct=6065

■ He modifies it to a nearby number
?acct=6066

■ Attacker views the victim's account information

Scenario 2



- Attacker notices the URL indicates his role
`/user/getAccounts`
- He modifies it to another directory (role)
`/admin/getAccounts`, or
`/manager/getAccounts`
- Attacker views more accounts than just their own

Cryptographic Failures



Cryptographic failures refer to problems with cryptography or the absence of cryptography altogether. Previously this item was known as Sensitive Data Exposure. Cryptographic failure may and often does lead to exposure of sensitive data. Such data typically include authentication details, personally identifiable information (PII) such as personal and financial information, health records, business secrets, and more.

Common cryptographic failures vulnerabilities include:

- Sensitive data transmitted (via HTTP, FTP, SMTP, etc) or stored in clear-text (in database, files, etc).
- Use of old or weak cryptographic algorithms.
- Use of weak or default encryption keys or re-use of compromised keys
- Encryption not enforced or server certificates not validated while communicating with it.

Injection:

An injection attack refers to untrusted data by an application that forces it to execute commands. Such data or malicious code is inserted by an attacker and can compromise data or the whole application. The most common injection attacks are SQL injections and cross-site scripting (XSS) attacks, but code injections, command injections, CCS injections, and others.

When an injection attack is successful, the attacker can view, modify or even delete data and possibly gain control over the server.

- User-supplied data is not validated, filtered, or sanitized by the application.
- Dynamic queries or non-parameterized calls without context-aware escaping are used directly in the interpreter.
- Hostile data is used within object-relational mapping (ORM) search parameters to extract additional, sensitive records.
- Hostile data is directly used or concatenated. The SQL or command contains the structure and malicious data in dynamic queries, commands, or stored procedures.





Insecure Design:

Insecure design is focused on the risks associated with **flaws in design and architecture**. A secure design can still have implementation defects leading to vulnerabilities that may be exploited. An insecure design cannot be fixed by a perfect implementation as by definition, needed security controls were never created to defend against specific attacks. One of the factors that contribute to insecure design is the **lack of business risk profiling** inherent in the software or system being developed, and thus the failure to determine what level of security design is required.

How to Prevent:

- Establish and use a secure development lifecycle with AppSec professionals to help evaluate and design security and privacy-related controls.
- OWASP Software Assurance Maturity Model (SAMM) helps to structure secure software development efforts.
- Use threat modeling for critical authentication, access control, business logic, and key flows.
- Segregate tenants robustly by design throughout all tiers
- Limit resource consumption by user or service

Security Misconfiguration

Security misconfiguration refers to security controls that are not secured or not configured correctly.



This vulnerability is frequently due to one of the following reasons:

- Lacking security hardening in any part of an application.
- Wrongly configured permissions on cloud services.
- Unnecessary features, such as ports, services, pages, accounts, or privileges are allowed or installed.
- Default accounts/passwords are enabled or unchanged.
- Error messages displayed to users contain stack traces or other sensitive information
- The latest security features are not enabled or implemented correctly
- The server, framework, libraries, or databases' security settings are not set to secure values
- Security headers or directives are not sent by the server or are not set to secure values
- The software is not up to date

Vulnerable and Outdated Components:

It is very common for web services to include a component with a known security vulnerability. When that happens it falls under this category.

The component with a known vulnerability could be the operating system itself, the CMS used, the web server, some plugin installed or even a library used by one of these plugins.

The most effective defense is continuous scanning of all code components for known vulnerabilities and deploying a patch or other remedy as quickly as possible when a vulnerability is detected



Identification and Authentication Failures:

When applications incorrectly execute functions related to session management or user authentication, intruders may be able to compromise passwords, security keys, or session tokens and permanently or temporarily assume the identities and permissions of other users.



How to Prevent:

- Enable and enforce Multi-Factor Authentication.
- Ensure that default passwords are only for one-time use and are updated as the user logs in.
- Enforce a password policy to prevent users from setting weak passwords.
- Brute-force protection should be implemented like use of CAPTCHA and rate limit on login page.

Software and Data Integrity Failures:

Software and data integrity failures relate to code and infrastructure that does not protect against integrity violations

For example, an application that relies on plugins, libraries, or modules from unverified and untrusted sources, repositories, or content delivery networks (CDNs) may be exposed to such a type of failure.



How to Prevent:

- Use digital signatures or similar mechanisms to verify the software or data is from the expected source and has not been altered.
- Ensure that there is a review process for code and configuration changes to minimize the chance that malicious code or configuration could be introduced into your software pipeline.
- Ensure that your CI/CD pipeline has proper segregation, configuration, and access control to ensure the integrity of the code flowing through the build and deploy processes.
- Ensure that unsigned or unencrypted serialized data is not sent to untrusted clients without some form of integrity check or digital signature to detect tampering or replay of the serialized data

```
<script src="https://example.com/example-framework.js" integrity="sha384-  
oqVuAfXRRKap7fdgcCY5uykM6+R9GqQ8K/uxy9rx7HNQlGYl1kPzQho1wx4JwY8wC"  
crossorigin="anonymous"></script>
```

Security Logging and Monitoring Failures:

Logging & monitoring are quite critical because it helps to detect, escalate, and respond to active breaches. and their absence or failures can directly impact visibility, incident alerting, and forensics.



Best Practice:

- Ensure all login, access control, and server-side input validation failures can be logged with sufficient user context to identify suspicious or malicious accounts and held for enough time to allow delayed forensic analysis.
- Ensure that logs are generated in a format that log management solutions can easily consume.
- Ensure log data is encoded correctly to prevent injections or attacks on the logging or monitoring systems.
- Ensure high-value transactions have an audit trail with integrity controls to prevent tampering or deletion, such as append-only database tables or similar.
- DevSecOps teams should establish effective monitoring and alerting such that suspicious activities are detected and responded to quickly.

Server-Side Request Forgery (SSRF):



SSRF is a web security flaw that allows an attacker to force a server-side application to send HTTP requests to any domain the attacker chooses.

When a web application fetches a remote resource without validating the user-supplied URL, an SSRF fault occurs. Even if the program is secured by a firewall, VPN, or another sort of network access control list, an attacker can force it to send a forged request to an unexpected location.

How to Prevent:

- All client-supplied input data must be sanitized and validated.
- Use a positive allow list to enforce the URL schema, port, and destination.
- Disable HTTP redirections.
- Only accept the intended IP address format (IPv4 or IPv6).
- To compare against the allow list, use the method/output library's value as the IP address.
- Validate incoming Domain Names.

OWASP MOBILE TOP 10 2016 RISKS



M1: Improper
Platform Usage

M2: Insecure
Data Storage

M3: Insecure
Communication

M4: Insecure
Authentication

M5: Insufficient
Cryptography

M6: Insecure
Authorization

M7: Client Code
Quality

M8: Code
Tampering

M9: Reverse
Engineering

M10: Extraneous
Functionality

API Security Top 10



Broken Object
Level
Authorization

Broken User
Authentication

Excessive Data
Exposure

Lack of
Resources &
Rate Limiting

Broken Function
Level
Authorization

Mass Assignment

Security
Misconfiguration

Injection

Improper Asset
Management

Insufficient
Logging and
Monitoring

OWASP IoT Top 10



11 Weak
Guessable, or
Hardcoded
Passwords

12 Insecure
Network Services

13 Insecure
Ecosystem
Interfaces

14 Lack of Secure
Update Mechanism

15 Use of Insecure
or Outdated
Components

16 Insufficient
Privacy Protection

17 Insecure Data
Transfer and
Storage

18 Lack of Device
Management

19 Insecure Default
Settings

110 Lack of
Physical Hardening

OWASP TOP PROJECTS

-
- OWASP Application Security Verification Standard (ASVS)
 - OWASP Top Ten
 - OWASP Security Testing Guide
 - OWASP Security Testing Checklist
 - OWASP Testing Labs
 - OWASP Tools
 - OWASP Cheat Sheet Series
 - OWASP Security Knowledge Framework
 - OWASP SAMM

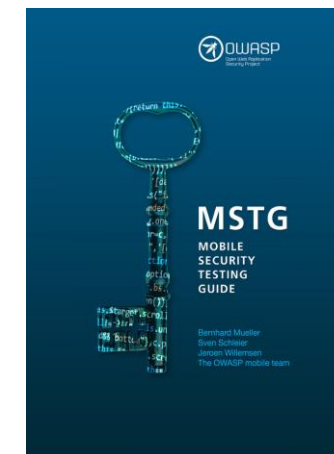
OWASP SECURITY TESTING GUIDE



The Security Testing Guide project produces the premier cybersecurity testing resource for various application like Web App, Mobile App, API's, Code Review, Vulnerability Management and Cloud Security.

Security testing guides are:

- OWASP Web Security Testing Guide (WSTG)
- Mobile Security Testing Guide (MSTG)
- OWASP Code Review Guide (OCRG)
- OWASP Vulnerability Management Guide (OVMG)
- OWASP Cloud Security Testing Guide (CSTG)



OWASP TOP PROJECTS

-
- OWASP Application Security Verification Standard (ASVS)
 - OWASP Top Ten
 - OWASP Security Testing Guide
 - **OWASP Security Testing Checklist**
 - OWASP Testing Labs
 - OWASP Tools
 - OWASP Cheat Sheet Series
 - OWASP Security Knowledge Framework
 - OWASP SAMM

OWASP SECURITY TESTING CHECKLIST



The pre-planned “to-do” list of tasks called a checklist.

OWASP team has created checklist which helps developer and testers for developing and testing the application.

OWASP Top checklist are:

- OWASP Web Application Penetration Checklist
- OWASP Mobile App Security Checklist
- OWASP Secure Coding Practices Checklist

OWASP Web Application Penetration Checklist



OWASP: Testing Guide v4.2 Checklist				
Information Gathering	Test Name	Objectives	Status	Notes
WSTG-INFO-01	Conduct Search Engine Discovery Reconnaissance for Information Leakage	- Identify what sensitive design and configuration information of the application, system, or organization is exposed directly (on the organization's website) or indirectly (via third-party services).	Not Started	
WSTG-INFO-02	Fingerprint Web Server	- Determine the version and type of a running web server to enable further discovery of any known vulnerabilities.	Not Started	
WSTG-INFO-03	Review Webserver Metafiles for Information Leakage	- Identify hidden or obfuscated paths and functionality through the analysis of metadata files. - Extract and map other information that could lead to better understanding of the systems at hand.	Not Started	
WSTG-INFO-04	Enumerate Applications on Webserver	- Enumerate the applications within scope that exist on a web server.	Not Started	
WSTG-INFO-05	Review Webpage Content for Information Leakage	- Review webpage comments and metadata to find any information leakage. - Gather JavaScript files and review the JS code to better understand the application and to find any information leakage. - Identify if source map files or other front-end debug files exist.	Not Started	
WSTG-INFO-06	Identify application entry points	- Identify possible entry and injection points through request and response analysis.	Not Started	
WSTG-INFO-07	Map execution paths through application	- Map the target application and understand the principal workflows.	Not Started	
WSTG-INFO-08	Fingerprint Web Application Framework	- Fingerprint the components being used by the web applications.	Not Started	
WSTG-INFO-09	Fingerprint Web Application		Not Started	
WSTG-INFO-10	Map Application Architecture	- Generate a map of the application at hand based on the research conducted.	Not Started	
Configuration and Deploy Management Testing	Test Name	Objectives	Status	Notes
WSTG-CONF-01	Test Network Infrastructure Configuration	- Review the applications' configurations set across the network and validate that they are not vulnerable. - Validate that used frameworks and systems are secure and not susceptible to known vulnerabilities due to unmaintained software or default settings and credentials.	Not Started	
WSTG-CONF-02	Test Application Platform Configuration	- Ensure that defaults and known files have been removed. - Validate that no debugging code or extensions are left in the production environments. - Review the logging mechanisms set in place for the application.	Not Started	



Testing Checklist

Summary Findings

Risk Assessment Calculator

References



OWASP Mobile App Security Checklist



Mobile Application Security Requirements - Android

ID	MSTG-ID	Detailed Verification Requirement	Level 1	Level 2	Status	Testing P
V1		Architecture, design and threat modelling				
1.1	MSTG-ARCH-1	All app components are identified and known to be needed.	✓	✓		Architectural Information
1.2	MSTG-ARCH-2	Security controls are never enforced only on the client side, but on the respective remote endpoints.	✓	✓		Injection Flaws (MSTG-ARCH-2 and MSTG-PLATFORM-2)
1.3	MSTG-ARCH-3	A high-level architecture for the mobile app and all connected remote services has been defined and security has been addressed in that architecture.	✓	✓		Architectural Information
1.4	MSTG-ARCH-4	Data considered sensitive in the context of the mobile app is clearly identified.	✓	✓		Identifying Sensitive Data
1.5	MSTG-ARCH-5	All app components are defined in terms of the business functions and/or security functions they provide.		✓	N/A	Environmental Information
1.6	MSTG-ARCH-6	A threat model for the mobile app and the associated remote services has been produced that identifies potential threats and countermeasures.		✓	N/A	Mapping the Application
1.7	MSTG-ARCH-7	All security controls have a centralized implementation.		✓	N/A	Testing for insecure Configuration of Instant Apps (MSTG-ARCH-1, MSTG-ARCH-7)
1.8	MSTG-ARCH-8	There is an explicit policy for how cryptographic keys (if any) are managed, and the lifecycle of cryptographic keys is enforced. Ideally, follow a key management standard such as NIST SP 800-57.		✓	N/A	Cryptographic policy
1.9	MSTG-ARCH-9	A mechanism for enforcing updates of the mobile app exists.		✓	N/A	Testing enforced updating (MSTG-ARCH-9)
1.10	MSTG-ARCH-10	Security is addressed within all parts of the software development lifecycle.		✓	N/A	Security Testing and the SDLC
1.11	MSTG-ARCH-11	A responsible disclosure policy is in place and effectively applied.		✓	N/A	
1.12	MSTG-ARCH-12	The app should comply with privacy laws and regulations.	✓	✓		
V2		Data Storage and Privacy				
2.1	MSTG-STORAGE-1	System credential storage facilities need to be used to store sensitive data, such as PII, user credentials or cryptographic keys.	✓	✓		Testing Local Storage for Sensitive Data (MSTG-STORAGE-1 and MSTG-STORAGE-2)
2.2	MSTG-STORAGE-2	No sensitive data should be stored outside of the app container or system credential storage facilities.	✓	✓		Testing Local Storage for Sensitive Data (MSTG-STORAGE-1 and MSTG-STORAGE-2)
2.3	MSTG-STORAGE-3	No sensitive data is written to application logs.	✓	✓		Testing Logs for Sensitive Data (MSTG-STORAGE-3)
2.4	MSTG-STORAGE-4	No sensitive data is shared with third parties unless it is a necessary part of the architecture.	✓	✓		Determining Whether Sensitive Data is Sent to Third Parties (MSTG-STORAGE-4)
2.5	MSTG-STORAGE-5	The keyboard cache is disabled on text inputs that process sensitive data.	✓	✓		Determining Whether the Keyboard Cache Is Disabled for Text Input Fields (MSTG-STORAGE-5)
2.6	MSTG-STORAGE-6	No sensitive data is exposed via IPC mechanisms.	✓	✓		Determining Whether Sensitive Stored Data Has Been Exposed via IPC Mechanisms (MSTG-STORAGE-6)
2.7	MSTG-STORAGE-7	No sensitive data, such as passwords or pins, is exposed through the user interface.	✓	✓		Checking for Sensitive Data Disclosure Through the User Interface (MSTG-STORAGE-7)
2.8	MSTG-STORAGE-8	No sensitive data is included in backups generated by the mobile operating system.		✓	N/A	Testing Backups for Sensitive Data (MSTG-STORAGE-8)
2.9	MSTG-STORAGE-9	The app removes sensitive data from views when moved to the background.		✓	N/A	Finding Sensitive Information in Auto-Generated Screenshots (MSTG-STORAGE-9)
2.10	MSTG-STORAGE-10	The app does not hold sensitive data in memory longer than necessary, and memory is cleared explicitly after use.		✓	N/A	Checking Memory for Sensitive Data (MSTG-STORAGE-10)
2.11	MSTG-STORAGE-11	The app enforces a minimum device-access-security policy, such as requiring the user to set a device passcode.		✓	N/A	Testing the Device-Access-Security Policy (MSTG-STORAGE-11)
2.12	MSTG-STORAGE-12	The app educates the user about the types of personally identifiable information processed, as well as security best practices the user should follow in using the app.		✓	N/A	Testing User Education (MSTG-STORAGE-12)
2.13	MSTG-STORAGE-13	No sensitive data should be stored locally on the mobile device. Instead, data should be retrieved from a remote endpoint when needed and only be kept in memory.		✓	N/A	
2.14	MSTG-STORAGE-14	If sensitive data is still required to be stored locally, it should be encrypted using a key derived from hardware backed storage		✓	N/A	

OWASP Secure Coding Practices Checklist



Secure Coding Practices Checklist

Input Validation:

- ☐ Conduct all data validation on a trusted system (e.g., The server)
- ☐ Identify all data sources and classify them into trusted and untrusted. Validate all data from untrusted sources (e.g., Databases, file streams, etc.)
- ☐ There should be a centralized input validation routine for the application
- ☐ Specify proper character sets, such as UTF-8, for all sources of input
- ☐ Encode data to a common character set before validating ([Canonicalize](#))
- ☐ All validation failures should result in input rejection
- ☐ Determine if the system supports UTF-8 extended character sets and if so, validate after UTF-8 decoding is completed
- ☐ Validate all client provided data before processing, including all parameters, URLs and HTTP header content (e.g. Cookie names and values). Be sure to include automated post backs from JavaScript, Flash or other embedded code
- ☐ Verify that header values in both requests and responses contain only ASCII characters
- ☐ Validate data from redirects (An attacker may submit malicious content directly to the target of the redirect, thus circumventing application logic and any validation performed before the redirect)
- ☐ Validate for expected data types
- ☐ Validate data range
- ☐ Validate data length
- ☐ Validate all input against a "white" list of allowed characters, whenever possible
- ☐ If any potentially [hazardous characters](#) must be allowed as input, be sure that you implement additional controls like output encoding, secure task specific APIs and accounting for the utilization of that data throughout the application . Examples of common hazardous characters include:
< > " ' % () & + \ ' \"
- ☐ If your standard validation routine cannot address the following inputs, then they should be checked discretely
 - o Check for null bytes (%00)
 - o Check for new line characters (%0d, %0a, \r, \n)
 - o Check for "dot-dot-slash" (../ or ..\) path alterations characters. In cases where UTF-8 extended character set encoding is supported, address alternate representation like: %c0%ae%c0%ae/ (Utilize [canonicalization](#) to address double encoding or other forms of obfuscation attacks)

OWASP TOP PROJECTS

-
- OWASP Application Security Verification Standard (ASVS)
 - OWASP Top Ten
 - OWASP Security Testing Guide
 - OWASP Security Testing Checklist
 - OWASP Testing Labs
 - OWASP Tools
 - OWASP Cheat Sheet Series
 - OWASP Security Knowledge Framework
 - OWASP SAMM

OWASP TESTING LABS



OWASP team has created vulnerable applications and VM. Just install and setup the labs and start hacking!

Testing labs are:

- OWASP Juice Shop
- WebGoat
- IoTGoat
- OWASP Broken Web Application(BWA)
- OWASP Vulnerable App
- And more...



OWASP TOP PROJECTS

-
- OWASP Application Security Verification Standard (ASVS)
 - OWASP Top Ten
 - OWASP Security Testing Guide
 - OWASP Security Testing Checklist
 - OWASP Testing Labs
 - **OWASP Tools**
 - OWASP Cheat Sheet Series
 - OWASP Security Knowledge Framework
 - OWASP SAMM

OWASP TOOLS



OWASP's mission is to help the world improve the security of its software. One of the best ways OWASP can do that is to help Open Source developers improve the software they are producing that everyone else relies on.

Top **open source** projects by OWASP :

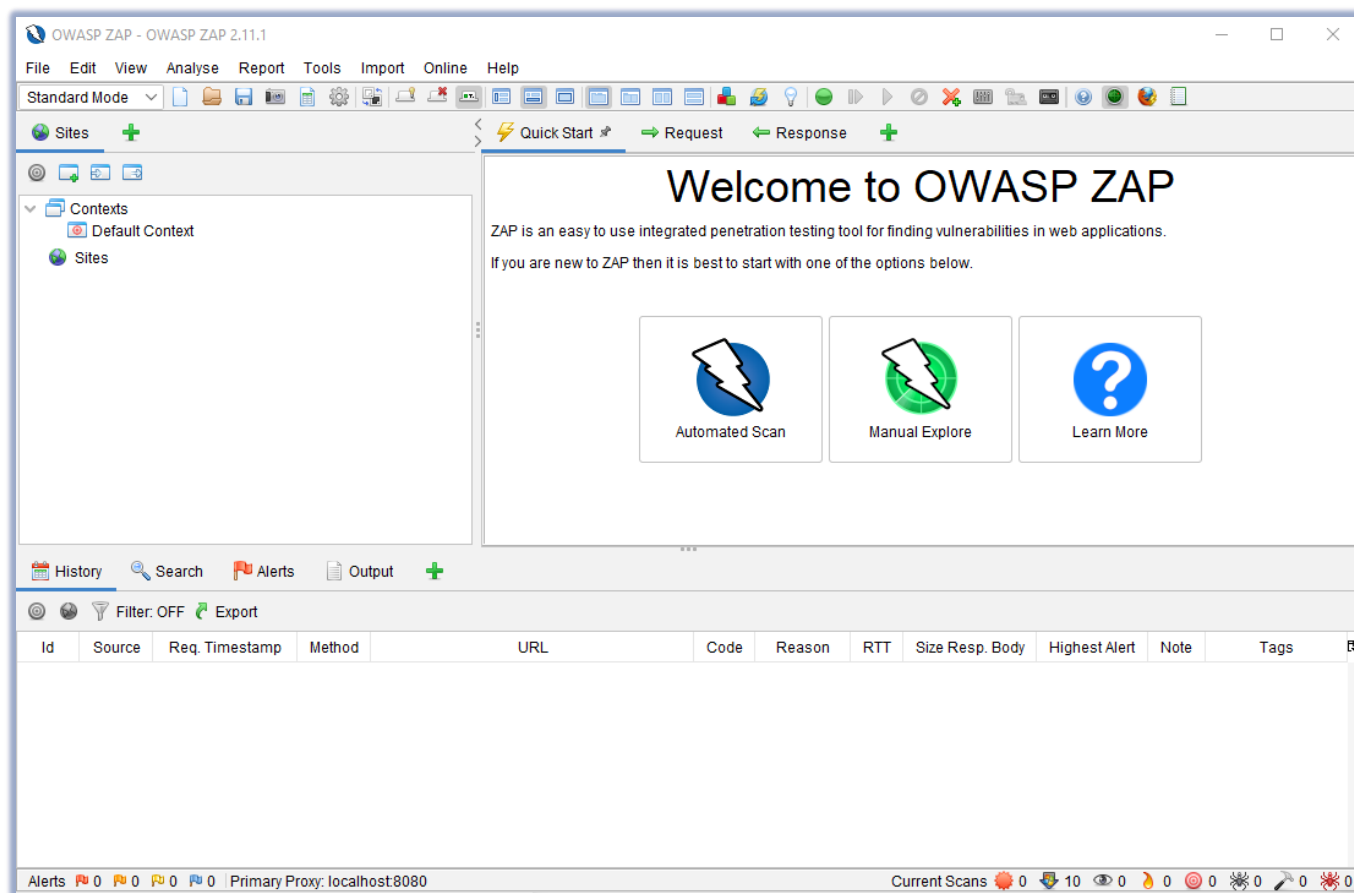
- OWASP Zed Attack Proxy (ZAP)
- OWASP Amass
- OWASP Dependency-Check
- OWASP Nightingale
- And more...





OWASP ZAP

Zed Attack Proxy (ZAP) is a free, open-source penetration testing tool being maintained under the umbrella of the Open Web Application Security Project (OWASP). ZAP is designed specifically for testing web applications and is both flexible and extensible.



Automate checking ASVS controls using ZAP scripts

Level 1 Automation

Source: <https://www.zaproxy.org/download/>



OWASP Amass

The OWASP Amass Project performs network mapping of attack surfaces and external asset discovery using open source information gathering and active reconnaissance techniques.

Information Gathering Techniques Used:

Technique	Data Sources
DNS	Brute forcing, Reverse DNS sweeping, NSEC zone walking, Zone transfers, FQDN alterations/permutations, FQDN Similarity-based Guessing
Scraping	AbuseIPDB, Ask, AskDNS, Baidu, Bing, DNSDumpster, DuckDuckGo, Gists, HackerOne, HyperStat, IPv4Info, PKey, RapidDNS, Riddler, Searchcode, Searx, SiteDossier, SpyOnWeb, Yahoo
Certificates	Active pulls (optional), Censys, CertSpotter, Crtsh, Digitorus, FacebookCT, GoogleCT
APIs	360PassiveDNS, ARIN, Ahrefs, AlienVault, AnubisDB, BinaryEdge, BGPView, BufferOver, BuiltWith, C99, Cloudflare, CommonCrawl, DNSDB, DNSlytics, Detectify,, GitHub, GitLab, Greynoise, HackerTarget, Hunter, IntelX, IPdata, IPinfo, Maltiverse, Mnemonic, N45HT, NetworksDB, and more.
Web Archives	Archivelt, Arquivo, HAW, UKWebArchive, Wayback

Source: <https://github.com/OWASP/Amass>



OWASP Dependency-Check

Dependency-Check is a Software Composition Analysis (SCA) tool that attempts to detect publicly disclosed vulnerabilities contained within a project's dependencies. It does this by determining if there is a Common Platform Enumeration (CPE) identifier for a given dependency. If found, it will generate a report linking to the associated CVE entries.

OWASP dependency-check's core analysis engine can be used as:

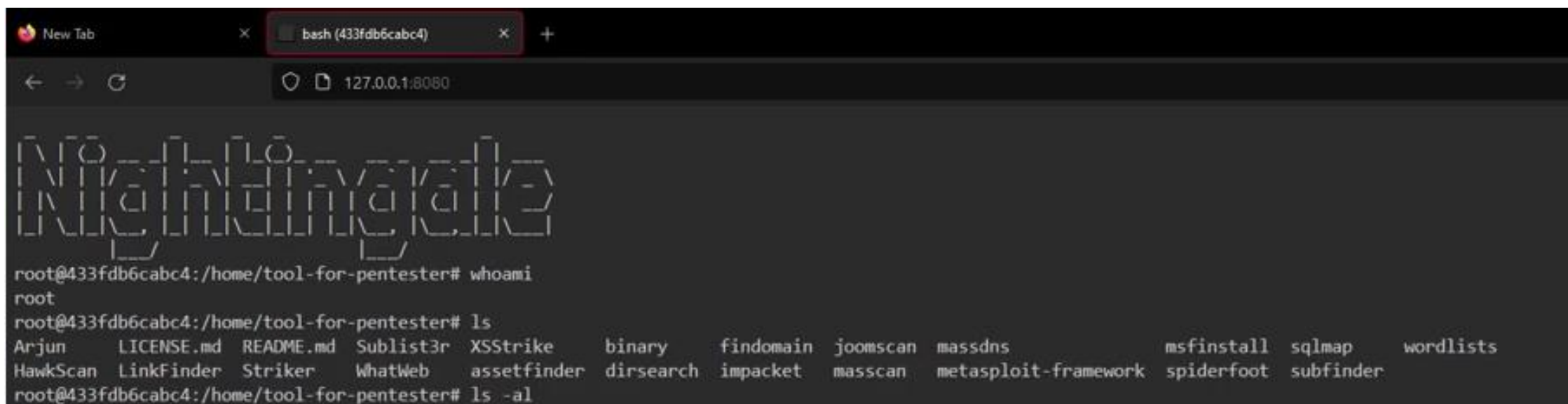
- Ant Task
- Command Line Tool
- Gradle Plugin
- Jenkins Plugin
- Maven Plugin - Maven 3.1 or newer required
- SBT Plugin

Source: <https://github.com/jeremylong/DependencyCheck>

OWASP Nightingale

Docker image for Pentesters.

Nightingale contain all the required famous tools that will be required to the pentester at the time of Penetration Testing. This docker image has a base support of debian and it is completely platform Independent.



```
bash (433fdb6cabc4)
127.0.0.1:8080
Nightingale
root@433fdb6cabc4:/home/tool-for-pentester# whoami
root
root@433fdb6cabc4:/home/tool-for-pentester# ls
Arjun    LICENSE.md  README.md  Sublist3r  XSSStrike  binary  findomain  joomscan  massdns  msfinstall  sqlmap  wordlists
HawkScan LinkFinder Striker    WhatWeb    assetfinder dirsearch impacket  masscan  metasploit-framework  spiderfoot  subfinder
root@433fdb6cabc4:/home/tool-for-pentester# ls -al
```

Source: <https://github.com/RAJANAGORI/Nightingale>

OWASP TOP PROJECTS

-
- OWASP Application Security Verification Standard (ASVS)
 - OWASP Top Ten
 - OWASP Security Testing Guide
 - OWASP Security Testing Checklist
 - OWASP Testing Labs
 - OWASP Tools
 - OWASP Cheat Sheet Series
 - OWASP Security Knowledge Framework
 - OWASP SAMM

OWASP CHEAT SHEET SERIES



The OWASP Cheat Sheet Series provides a concise collection of high value information on specific application security topics. These cheat sheets were created by various application security professionals who have expertise in specific topics.

OWASP Cheat Sheet Series

[Introduction](#)

[Index Alphabetical](#)

[Index ASVS](#)

[Index Proactive Controls](#)

[Index Top 10](#)

Cheatsheets

[AJAX Security](#)

[Abuse Case](#)

[Access Control](#)

[Attack Surface Analysis](#)

[Authentication](#)

[Authorization](#)

[Authorization Testing](#)

[Automation](#)

[PHP Configuration](#)

[Password Storage](#)

[Pinning](#)

[Query Parameterization](#)

[REST Assessment](#)

[REST Security](#)

[Ruby on Rails](#)

[SAML Security](#)

[SQL Injection Prevention](#)

[Securing Cascading Style Sheets](#)



Source: <https://cheatsheetseries.owasp.org/index.html>

OWASP TOP PROJECTS

-
- OWASP Application Security Verification Standard (ASVS)
 - OWASP Top Ten
 - OWASP Security Testing Guide
 - OWASP Security Testing Checklist
 - OWASP Testing Labs
 - OWASP Tools
 - OWASP Cheat Sheet Series
 - OWASP Security Knowledge Framework
 - OWASP SAMM

OWASP SECURITY KNOWLEDGE FRAMEWORK



Security Knowledge Framework (SKF) is an expert system application that uses the OWASP Application Security Verification Standard with detailed code examples (secure coding principles) to help developers in pre-development and post-development phases and **create applications that are secure by design**.

OWASP-SKF is a fully open-source Python-Flask web Application that uses the OWASP ASVS to train developers and testers for building and securing the application.

SKF FEATURES :

- Create project and start gathering requirement for new features/sprints.
- Code example, It provides best practices to keep the project secure.
- Checklists, It includes ASVS and MASVS checklist and also can create custom checklist based on features implemented in the project.
- Labs, It include OWASP Juice-shop labs to improve testing skills and know about various vulnerabilities.
- And more...

SKF FEATURES

SECURE CODING STARTS HERE.



PROJECTS

Create projects in SKF and start gathering requirements for your features/sprints



CODE EXAMPLES

An extensive library of common hacks, exploits, and best practices. Learn the hacker mindset and keep your project secure..



CHECKLISTS

Out of the box SKF comes with ASVS and MASVS included.



LABS

Train your hacking skills with over 50+ interactive labs that you can run locally or through the SKF UI in your Kubernetes cluster.



KNOWLEDGE BASE

All requirements are correlated to knowledgebase items to give you more in depth information about attack vectors, impact, mitigation and best practices.



USER MANAGEMENT

Manage your users by adding linking SKF to your favourite OIDC provider



DESIGN PATTERNS

We included the most used user-stories in SKF to get your team get started quickly implementing ASVS in your projects.



SUPPORT

Find us on our Gitter channel to ask us anything about SKF and how to get yourself started.



SKF DEMO : <https://demo.securityknowledgeframework.org/>



 Security Knowledge Framework

Search...

Logout

DashboardManage ProjectsCode ExamplesChecklistsKnowledgebaseLabs

DASHBOARDWelcome to Dashboard

Security Knowledge Framework

OWASP-SKF is an open source web application that explains secure coding principles in multiple programming languages. The goal of OWASP-SKF is to help you learn and integrate security by design in your software development and build applications that are secure by design. OWASP-SKF does this through manageable software development projects with checklists (using OWASP-ASVS/OWASP-MASVS or custom security checklists) and labs to practise security verification (using SKF-Labs, OWASP Juice-shop, and best practice code examples from SKF and the OWASP-Cheatsheets). If you want a simple set of courses on the fundamentals of developing secure software that don't involve programming exercises, you might instead consider the Secure Software Development Fundamentals Courses available on edX from OpenSSF.

Start tour



CODE EXAMPLES

The code examples show how to approach some common functionalities and how to apply the



CHECKLIST

Show different checklists that you can select for reference material. Click on the security control in the checklist to



KNOWLEDGE BASE

All the information which is correlated throughout the project can be found here! Need specific information on the



SKF LABS

We created SKF Labs for Developers and security specialist to train their security testing skills. Also provided are

CODE Examples



demo.securityknowledgeframework.org/code-example/view

 Security Knowledge Framework

[Dashboard](#) [Manage Projects](#) [Code Examples](#) [Checklists](#) [Knowledgebase](#) [Labs](#) [Logout](#)

CODE EXAMPLE Code Examples / View

Web applications

Mobile applications

Custom checklist

Web applications

 **asp-needs-reviewing - Single input validation control** [Read more](#)

 **asp-needs-reviewing - Audit Logs** [Read more](#)

```
1 Audit logs
2
3
4 Example:
5
6
7     using System;
8     using System.Collections.Generic;
9     using System.Linq;
10    using System.Web;
```

Checklists



demo.securityknowledgeframework.org/checklists/view

 Security Knowledge Framework

Search...



Logout

 Dashboard

 Manage Projects

 Code Examples

 Checklists

 Knowledgebase

 Labs

CHECKLISTS

Checklists / View

Web applications

Mobile applications

Custom checklist

Search Checklist

Architecture, Design and Threat Modeling Requirements

Security architecture has almost become a lost art in many organizations. The days of the enterprise architect have passed in the age of DevSecOps. The application security field must catch up and adopt agile security principles while re-introducing leading security architecture principles to software practitioners. Architecture is not an implementation, but a way of thinking about a problem that has potentially many different answers, and no one single "correct" answer. All too often, security is seen as inflexible and demanding that developers fix code in a particular way, when the developers may know a much better way to solve the problem. There is no single, simple solution for architecture, and to pretend otherwise is a disservice to the software engineering field.

Active: True

Inspect checklist

Authentication Verification Requirements

Authentication is the act of establishing, or confirming, someone (or something) as authentic and that claims made by a person or about a device are correct, resistant to impersonation, and prevent recovery or interception of passwords.

Knowledge Base



 Security Knowledge Framework

Search...

Logout

Dashboard

Manage Projects

Code Examples

Checklists

Knowledgebase

Labs

KNOWLEDGEBASE

Knowledgebase / View

Web applications

Mobile applications

Custom checklist

Web applications

Search KB i

J2EE Misconfiguration Data Transmission Without Encryption

Read More

Description:

Information sent over a network can be compromised while in transit. An attacker may be able to read or modify the contents if the data are sent in plaintext or are weakly encrypted.

Mitigation:

PHASE: System Configuration:
The application configuration should ensure that SSL or an encryption mechanism of equivalent strength and vetted reputation is used for all access-controlled pages.

Labs



demo.securityknowledgeframework.org/labs/view

 Security Knowledge Framework

Search...



Logout

Dashboard

Manage Projects

Code Examples

Checklists

Knowledgebase

Labs

LABS

Labs / View

Red & Blue Labs

Search Lab

#	Name	Label	Level	Status	Write-up	Action
	Path traversal (LFI)	SKF-labs	1		Click here	Start
	Cross Site Scripting	SKF-labs	1		Click here	Start
	Cross site scripting (attribute)	SKF-labs	1		Click here	Start
	Cross site scripting (href)	SKF-labs	1		Click here	Start
	Insecure file upload	SKF-labs	1		Click here	Start

Labs – Start a Lab



demo.securityknowledgeframework.org/labs/view

Security Knowledge Framework

Search...

Dashboard Manage Projects Code Examples Checklists Knowledgebase Labs

LABS

Red & Blue Labs

Search Lab

Lab deployment URL

http://xss-775496.securityknowledgeframework-labs.org

Close

#	Name	Status	Write-up	Action
	Path traversal (LFI)		Click here	Start
	Cross Site Scripting		Click here	Start
	Cross site scripting (attribute)	SKF-labs	1	Click here Start
	Cross site scripting (href)	SKF-labs	1	Click here Start
	Insecure file upload	SKF-labs	1	Click here Start
	Clickjacking	SKF-labs	1	Click here Start

Labs – Demonstration



← → ↻ ⚠ Not secure | xss-775496.securityknowledgeframework-labs.org



OWASP S.K.F.

chat on [gitter](#)

Hint 0/2 ([show](#))

Live demonstration!

XSS!

[Submit Button](#)

Labs – Demonstration - Hints



← → ↻ ⚠ Not secure | xss-775496.securityknowledgeframework-labs.org/home



OWASP S.K.F.

chat on [gitter](#)

Hint 2/2 ([show](#))

1. Find, identify and explore every possible input field that could be object of injection.
2. Try to inject and execute javascript code in the field.

Live demonstration!

XSS!

[Submit Button](#)

Labs – Demonstration – Inject XSS



← → ↻ ⚠ Not secure | xss-775496.securityknowledgeframework-labs.org/home



OWASP S.K.F.

chat on 

Live demonstration!

XSS!

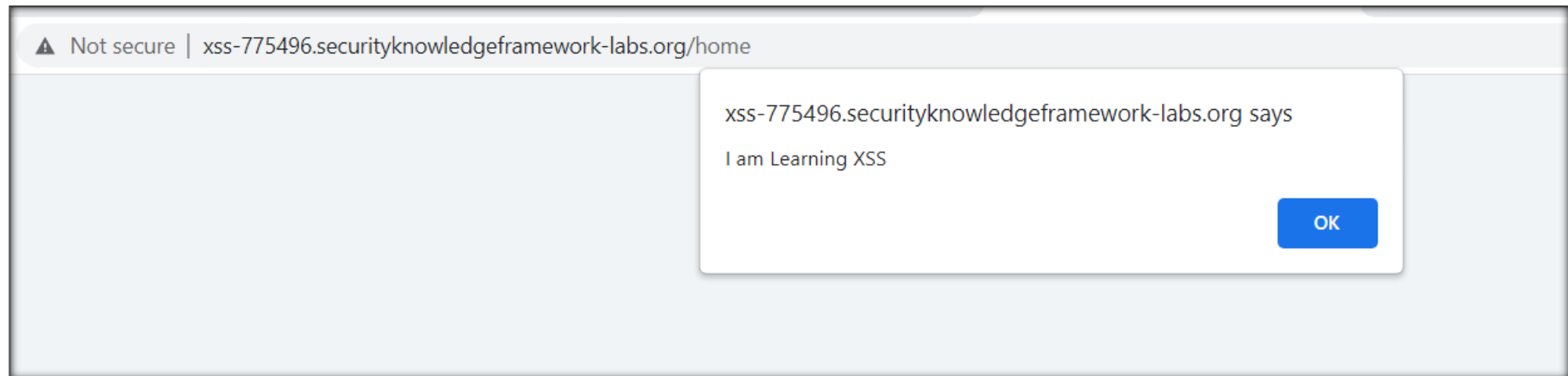
```
<script>alert("I am Learning XSS")</script>
```

Submit Button

Hint 2/2 (show)

1. Find, identify and explore every possible input field that could be object of injection.
2. Try to inject and execute javascript code in the field.

Labs – Demonstration – Exploit



Labs - More Labs



demo.securityknowledgeframework.org/labs/view

Security Knowledge Framework

Search...

Logout

Dashboard Manage Projects Code Examples Checklists Knowledgebase Labs

	Admin Section	juice-shop	2	Click here	Start
	Deprecated Interface	juice-shop	2	Click here	Start
	Five-Star Feedback	juice-shop	2	Click here	Start
	Login Admin	juice-shop	2	Click here	Start
	Login MC SafeSearch	juice-shop	2	Click here	Start
	Password Strength	juice-shop	2	Click here	Start
	Reflected XSS	juice-shop	2	Click here	Start
	Security Policy	juice-shop	2	Click here	Start

→ ↻ github.com/blabla1337/skf-flask



Product ▾ Team Enterprise Explore ▾ Marketplace Pricing ▾

Search

blabla1337 / skf-flask Public

Code Issues 44 Pull requests 1 Actions Projects 2 Wiki Security Insights

main 8 branches 10 tags

Go to file

Code ▾



blabla1337 Merge pull request #753 from MeVitae/k8s-dabase-update

✓ 040eaf2 6 days ago 3,271 commits

📁 .github/workflows	Create greetings.yml	10 months ago
📁 Angular2	small hot fixes because Angular	6 months ago
📁 Docker	Merge pull request #753 from MeVitae/k8s-dabase-update	6 days ago
📁 cover	added API's for code labs, added unit tests, egg	12 months ago
📁 external_resources	fixed Levels maturity in overviews	2 years ago
📁 installations	Merge pull request #753 from MeVitae/k8s-dabase-update	6 days ago
📁 skf	Update initial_data.py	2 months ago
📁 tests	Update e2e_tests.py	9 months ago
📄 .codecov.yml	merge	4 years ago
📄 .gitignore	fixes	17 months ago
📄 .scrutinizer.yml	Fixed the updatedb command, was bugged	4 years ago
📄 .travis.yml	update	17 months ago
📄 CONTRIBUTING.md	corrected the commands and a few mistakes	3 months ago
📄 LICENSE	ASVS legenda added and color codes added in ASVS checklist (#443)	4 years ago

SETUP SKF AT YOUR PREMISES

OWASP TOP PROJECTS

-
- OWASP Application Security Verification Standard (ASVS)
 - OWASP Top Ten
 - OWASP Security Testing Guide
 - OWASP Security Testing Checklist
 - OWASP Testing Labs
 - OWASP Tools
 - OWASP Cheat Sheet Series
 - OWASP Security Knowledge Framework
 - OWASP SAMM

OWASP SAMM



SAMM stands for Software Assurance Maturity Model. SAMM is a prescriptive model, an open framework which is simple to use, fully defined, and measurable. The solution details are easy enough to follow even for non-security personnel. It helps organizations **analyze their current software security practices**, **build** a security program in defined iterations, show **progressive improvements** in secure practices, and define and **measure security-related activities**.

The structure and setup of the OWASP SAMM maturity model are made to support:

1. The **assessment** of the current software assurance posture,
2. The definition of the **strategy** (i.e. the target) that the organization should implement,
3. The formulation of an implementation **roadmap** of how to get there, and
4. Prescriptive advice on how to **implement** particular activities.

HOW TO APPLY



PREPARE

ASSESS

SET THE TARGET

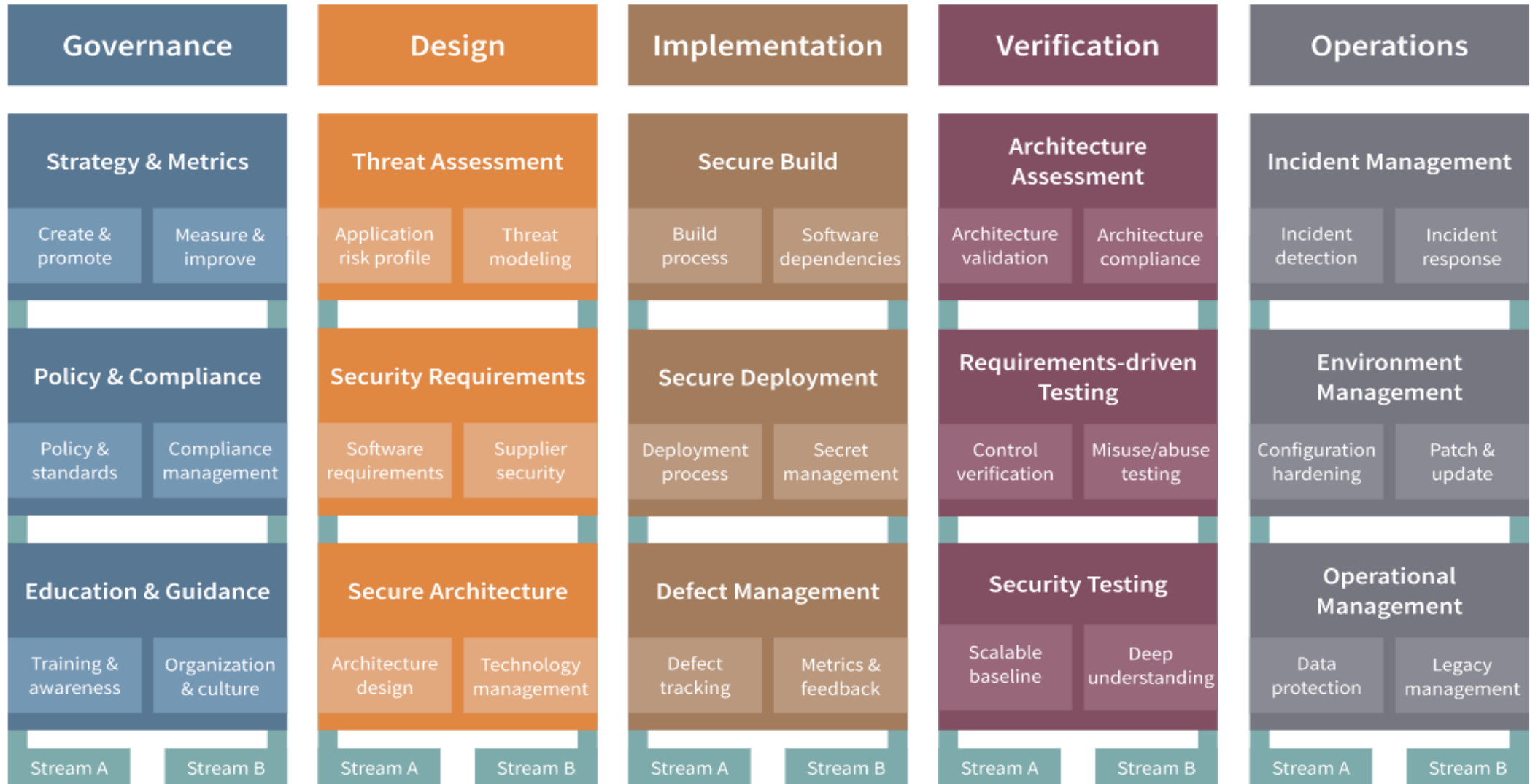
DEFINE THE PLAN

IMPLEMENT

ROLL OUT

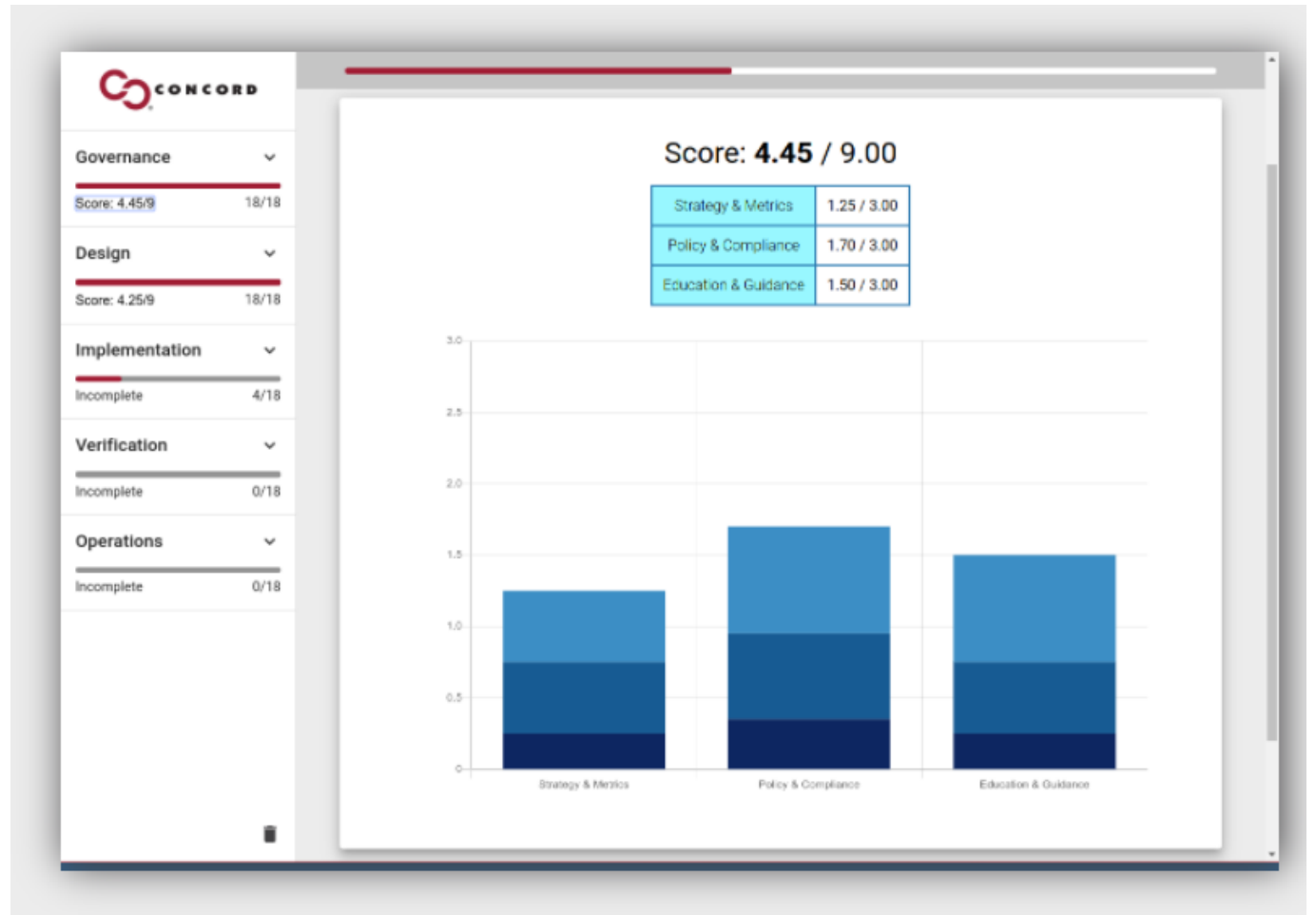
1. Ensure a proper start of the project
2. Identify and understand the maturity of your chosen scope in each of the 15 software security practices
3. Develop a target score that you can use as a measuring stick to guide you to act on the most important activities for your situation
4. Develop or update your plan to take your organization to the next level
5. Work the plan
6. Ensure that improvements are available and effectively used within the organization

SAMM Model:



ASSESS YOUR APPLICATION MATURITY LEVEL

- <https://owasp samm.org/assessment/>
- <https://concordusa.com/SAMM/>



REFERENCE



OWASP.ORG





Questions?



THANK YOU

indravenik@cdac.in